

# Laboratório de Ciência de Dados: Classificação de Textos em Língua Portuguesa

Nome: Abimael Gomes Mesquita Filho  
Nome: Kaue Cruz Gomes

Matrícula: 583351  
Matrícula: 585384

## 1. Introdução

O Processamento de Linguagem Natural (PLN) é uma área da Ciência de Dados voltada para o tratamento e a análise de informações expressas em linguagem humana. Por meio de técnicas de pré-processamento, representação numérica dos textos e algoritmos de aprendizado de máquina, é possível identificar padrões e realizar tarefas como classificação, agrupamento e detecção de comportamentos anômalos.

Neste projeto foi utilizado um conjunto de dados contendo textos em língua portuguesa relacionados à classificação de discursos de ódio. O objetivo foi desenvolver um pequeno pipeline de Processamento de Linguagem Natural, envolvendo o tratamento dos textos, a análise exploratória do corpus, a transformação dos textos em representações numéricas e a aplicação de modelos de classificação supervisionada.

Foram utilizados dois algoritmos de classificação: Naive Bayes e Regressão Logística. Para representar os textos numericamente, foram comparadas duas abordagens: Bag-of-Words, utilizando o CountVectorizer, e TF-IDF. Além disso, foi utilizado o SMOTE no conjunto de treinamento para tratar o desbalanceamento entre as classes.

Ao final, os modelos foram avaliados por meio de acurácia, precisão (precision), revocação (recall) e F1-score, permitindo comparar o comportamento dos classificadores nas diferentes representações dos textos.

---

## 2. Pré-processamento e Limpeza Textual

O pré-processamento é uma etapa fundamental em tarefas de Processamento de Linguagem Natural, pois os algoritmos de aprendizado de máquina não trabalham diretamente com textos em sua forma original. Dessa forma, os dados precisam ser tratados e posteriormente transformados em representações numéricas.

O conjunto de dados utilizado possui 5.670 instâncias e apresenta, entre suas principais colunas, a coluna `text`, contendo os textos, e a coluna `hatespeech_comb`, utilizada como

variável de classe. O dataset também possui informações referentes às classificações individuais dos anotadores.

A etapa de pré-processamento textual foi aplicada antes da transformação dos textos em vetores numéricos. O objetivo foi reduzir ruídos e tornar os textos mais adequados para a etapa de classificação.

## **2.1 Normalização**

A normalização consiste em padronizar os textos para diminuir diferenças que não contribuem para a classificação. Entre os procedimentos normalmente utilizados nessa etapa estão a conversão dos caracteres para letras minúsculas e a padronização da escrita.

Essa etapa é importante porque palavras escritas com letras maiúsculas e minúsculas podem ser interpretadas como tokens diferentes. Ao normalizar os textos, reduz-se a quantidade de variações desnecessárias no vocabulário.

## **2.2 Remoção de ruídos**

Os textos presentes no corpus podem conter elementos que não representam diretamente o conteúdo linguístico utilizado na classificação, como menções a usuários, caracteres especiais, pontuação, quebras de linha e outros elementos provenientes de mensagens de redes sociais.

A remoção desses ruídos contribui para reduzir a dimensionalidade da representação dos textos e evita que elementos pouco relevantes tenham influência excessiva sobre os modelos.

## **2.3 Tokenização**

A tokenização consiste em dividir cada texto em unidades menores, chamadas tokens, normalmente representadas por palavras ou termos.

Por exemplo, um texto como:

"Os usuários discutiram sobre o problema"

pode ser dividido em tokens como:

"Os", "usuários", "discutiram", "sobre", "o", "problema".

Essa transformação permite que os algoritmos de representação vetorial contabilizem a ocorrência dos termos presentes no corpus.

## **2.4 Lematização ou stemming**

A lematização e o stemming têm como objetivo reduzir diferentes formas de uma palavra a uma representação comum.

Por exemplo, palavras como "estudar", "estudando" e "estudou" podem apresentar relação morfológica. A redução dessas variações pode diminuir o tamanho do vocabulário e facilitar a identificação de padrões.

A utilização desse tipo de tratamento deve ser considerada de acordo com o comportamento do corpus, pois uma redução excessiva pode eliminar informações importantes para a classificação.

## 2.5 Tratamento do desbalanceamento

Foi identificado um desbalanceamento entre as classes. A classe 0 possui 3.882 exemplos, enquanto a classe 1 possui 1.788 exemplos. Portanto, a classe 0 representa a maioria dos dados.

Para reduzir os efeitos desse desequilíbrio, foi utilizado o método SMOTE durante o treinamento dos modelos. O SMOTE cria exemplos sintéticos da classe minoritária a partir de exemplos existentes, buscando tornar o conjunto de treinamento mais equilibrado.

É importante destacar que o balanceamento foi aplicado ao conjunto de treinamento, evitando que informações do conjunto de teste sejam utilizadas durante o processo de treinamento dos modelos.

---

# 3. Análise Exploratória do Corpus

A análise exploratória teve como objetivo compreender as principais características do conjunto de dados antes da construção dos modelos de classificação.

## 3.1 Tamanho do corpus e número de classes

O conjunto de dados possui:

- Número total de textos: 5.670;
- Número de classes: 2;
- Classe 0: 3.882 textos;
- Classe 1: 1.788 textos.

Considerando o total de 5.670 registros, aproximadamente 68,47% pertencem à classe 0 e 31,53% pertencem à classe 1.

| Class<br>e | Quantidade | Percentual<br>aproximado |
|------------|------------|--------------------------|
| 0          | 3.882      | 68,47%                   |

|              |              |             |
|--------------|--------------|-------------|
| 1            | 1.788        | 31,53%      |
| <b>Total</b> | <b>5.670</b> | <b>100%</b> |

A diferença entre as quantidades demonstra a existência de desbalanceamento no corpus, justificando a aplicação de uma técnica como o SMOTE durante o treinamento.

## 3.2 Distribuição das classes

A distribuição das classes foi representada graficamente por meio de um gráfico de contagem. O gráfico evidencia que a classe 0 possui quantidade significativamente maior de exemplos que a classe 1.

Essa diferença é relevante porque um modelo treinado diretamente sobre dados desbalanceados pode apresentar maior facilidade para reconhecer a classe majoritária e maior dificuldade para identificar corretamente exemplos pertencentes à classe minoritária.

Por esse motivo, além da acurácia, foram consideradas métricas como precision, recall e F1-score, principalmente para observar o comportamento dos modelos na identificação da classe 1.

## 3.3 Frequência de palavras

Também foi realizada uma análise da frequência das palavras presentes no corpus. Essa análise permite identificar os termos que aparecem com maior frequência nos textos e auxilia na compreensão inicial das características linguísticas dos dados.

A visualização das palavras mais frequentes foi realizada por meio de um gráfico de barras. Essa abordagem é útil porque apresenta de forma direta os termos mais recorrentes no corpus.

Além disso, uma nuvem de palavras pode ser utilizada como representação visual complementar, permitindo observar de maneira rápida os termos predominantes no conjunto de textos.

## 3.4 Aplicações de aprendizado não supervisionado

Após a conversão dos textos para uma representação numérica, existem diversas possibilidades de aplicação de técnicas de aprendizado não supervisionado.

### Modelagem de tópicos com LDA

A primeira possibilidade é utilizar Latent Dirichlet Allocation (LDA) para descobrir temas latentes presentes nos textos. O método poderia agrupar palavras que aparecem frequentemente em determinados documentos e, a partir disso, identificar possíveis assuntos presentes no corpus.

## Clusterização com K-Means

A segunda possibilidade seria aplicar K-Means para agrupar os textos de acordo com sua similaridade. Nesse caso, cada documento seria representado por um vetor numérico e os textos mais semelhantes seriam direcionados para os mesmos grupos.

Isso poderia permitir a identificação de diferentes padrões de conteúdo sem utilizar previamente os rótulos das classes.

## Detecção de anomalias com Isolation Forest

Uma terceira possibilidade seria utilizar Isolation Forest para identificar textos que apresentam características muito diferentes do restante do corpus.

Essa abordagem poderia ser utilizada para encontrar possíveis mensagens atípicas, padrões incomuns ou exemplos que merecessem uma análise individual.

Dessa forma, as três possibilidades identificadas são:

1. Modelagem de tópicos com LDA;
  2. Clusterização utilizando K-Means;
  3. Detecção de anomalias utilizando Isolation Forest.
- 

# 4. Construção dos Modelos de Classificação

Para a classificação dos textos foram escolhidos dois modelos diferentes: Naive Bayes e Regressão Logística.

A escolha desses modelos ocorreu porque ambos são adequados para problemas de classificação de documentos representados por vetores esparsos, além de permitirem uma comparação entre abordagens diferentes.

## 4.1 Naive Bayes

O Naive Bayes é um classificador probabilístico baseado no Teorema de Bayes.

O modelo calcula a probabilidade de uma determinada classe considerando as características observadas no texto. Sua principal característica é assumir independência entre as características utilizadas no processo de classificação.

Em problemas de classificação textual, o Naive Bayes apresenta como vantagem a simplicidade computacional e o bom desempenho em representações como Bag-of-Words.

Neste projeto, o modelo foi utilizado tanto com CountVectorizer quanto com TF-IDF.

## 4.2 Regressão Logística

A Regressão Logística é um algoritmo de classificação que estima a probabilidade de uma instância pertencer a determinada classe.

Para classificação binária, o modelo utiliza uma função logística para transformar uma combinação das características em uma probabilidade entre 0 e 1.

Sua utilização é bastante comum em classificação de textos porque consegue trabalhar com grande quantidade de características e representações esparsas, como as produzidas pelo TF-IDF.

Neste projeto, a Regressão Logística também foi aplicada utilizando tanto CountVectorizer quanto TF-IDF.

---

## 5. Representação das Features

Foram utilizadas duas técnicas de representação dos textos: Bag-of-Words, por meio do CountVectorizer, e TF-IDF.

### 5.1 Bag-of-Words – CountVectorizer

O CountVectorizer representa cada documento de acordo com a quantidade de vezes que cada palavra aparece.

Por exemplo, considerando um vocabulário com as palavras:

"casa", "carro" e "cidade",

um documento que contenha duas ocorrências de "casa", uma de "carro" e nenhuma de "cidade" poderia ser representado pelo vetor:

[2, 1, 0]

A principal vantagem dessa representação é a simplicidade. Entretanto, ela considera principalmente a frequência absoluta das palavras e não leva em conta diretamente o quanto determinado termo é comum ou raro em todo o corpus.

### 5.2 TF-IDF

O TF-IDF procura atribuir maior importância a palavras que são relevantes para determinado documento e aparecem menos frequentemente no restante do corpus.

A medida considera dois componentes principais:

TF, que representa a frequência do termo no documento, e IDF, que representa a importância do termo considerando sua ocorrência nos documentos.

De forma simplificada:

$$\text{TF-IDF} = \text{TF} \times \text{IDF}$$

A utilização do TF-IDF pode ser vantajosa em classificação textual porque palavras muito comuns em diversos documentos tendem a receber menor peso, enquanto termos mais específicos podem receber maior importância.

---

## 6. Treinamento e Avaliação dos Modelos

Foram realizados quatro experimentos principais:

1. Naive Bayes + CountVectorizer;
2. Regressão Logística + CountVectorizer;
3. Naive Bayes + TF-IDF;
4. Regressão Logística + TF-IDF.

Em todos os experimentos foi aplicado SMOTE ao conjunto de treinamento para tratar o desbalanceamento das classes.

Os resultados obtidos foram avaliados utilizando acurácia, precision, recall e F1-score.

---

## 7. Resultados Quantitativos

Os resultados obtidos para os quatro experimentos são apresentados a seguir.

| Modelo              | Representação   | Acurácia | F1 classe<br>0 | F1 classe<br>1 |
|---------------------|-----------------|----------|----------------|----------------|
| Naive Bayes         | CountVectorizer | 0,7240   | 0,79           | 0,60           |
| Regressão Logística | CountVectorizer | 0,6772   | 0,75           | 0,55           |
| Naive Bayes         | TF-IDF          | 0,6658   | 0,72           | 0,58           |

|                     |        |        |      |      |
|---------------------|--------|--------|------|------|
| Regressão Logística | TF-IDF | 0,7319 | 0,81 | 0,57 |
|---------------------|--------|--------|------|------|

Os resultados mostram que a maior acurácia foi obtida pela Regressão Logística utilizando TF-IDF, com aproximadamente 73,19%.

Em segundo lugar ficou o Naive Bayes utilizando CountVectorizer, com aproximadamente 72,40%.

A Regressão Logística com CountVectorizer apresentou acurácia de aproximadamente 67,72%, enquanto o Naive Bayes com TF-IDF apresentou aproximadamente 66,58%.

---

## 8. Naive Bayes com CountVectorizer

O Naive Bayes utilizando Bag-of-Words apresentou acurácia de 72%.

Para a classe 0, foram obtidos:

- Precision: 0,83;
- Recall: 0,76;
- F1-score: 0,79.

Para a classe 1:

- Precision: 0,55;
- Recall: 0,65;
- F1-score: 0,60.

O resultado demonstra que o modelo apresentou desempenho melhor na identificação da classe 0 do que na classe 1.

A diferença entre os F1-scores das duas classes indica que, apesar do balanceamento realizado no treinamento, ainda existe maior dificuldade na identificação correta da classe 1.

---

## 9. Regressão Logística com CountVectorizer

A Regressão Logística utilizando CountVectorizer apresentou acurácia de aproximadamente 67,72%.

Para a classe 0:

- Precision: 0,81;
- Recall: 0,70;
- F1-score: 0,75.

Para a classe 1:

- Precision: 0,49;
- Recall: 0,64;
- F1-score: 0,55.

Comparando com o Naive Bayes utilizando a mesma representação, a Regressão Logística apresentou desempenho inferior.

A diferença é observada principalmente na acurácia, que caiu de aproximadamente 72,40% para 67,72%, e no F1-score da classe 1, que passou de 0,60 para 0,55.

---

## 10. Naive Bayes com TF-IDF

Quando o Naive Bayes foi combinado com TF-IDF, a acurácia foi de aproximadamente 66,58%.

Para a classe 0 foram obtidos:

- Precision: 0,84;
- Recall: 0,64;
- F1-score: 0,72.

Para a classe 1:

- Precision: 0,48;
- Recall: 0,73;
- F1-score: 0,58.

Um ponto importante nesse resultado é o recall da classe 1, que chegou a 0,73. Isso significa que, entre os exemplos que realmente pertencem à classe 1, o modelo conseguiu identificar uma parcela maior.

Por outro lado, a precision da classe 1 foi de apenas 0,48, indicando que uma quantidade considerável das previsões positivas estava incorreta.

---

## 11. Regressão Logística com TF-IDF

A melhor combinação encontrada nos experimentos foi a Regressão Logística com TF-IDF.

O modelo apresentou:

- Acurácia: 0,7319;
- Precision da classe 0: 0,80;
- Recall da classe 0: 0,81;
- F1-score da classe 0: 0,81;
- Precision da classe 1: 0,58;
- Recall da classe 1: 0,56;
- F1-score da classe 1: 0,57.

Esse resultado mostra que a representação TF-IDF foi particularmente adequada para a Regressão Logística neste conjunto de dados.

Entretanto, mesmo sendo o melhor modelo em acurácia, seu desempenho na classe 1 ainda é inferior ao desempenho observado na classe 0. Isso demonstra que a tarefa de identificar corretamente os textos pertencentes à classe minoritária continua sendo um desafio.

---

## 12. Comparação entre CountVectorizer e TF-IDF

A comparação entre as duas representações mostra que não existe uma representação universalmente melhor para todos os modelos.

No caso do Naive Bayes, o CountVectorizer apresentou desempenho superior ao TF-IDF:

- Naive Bayes + CountVectorizer: 72,40%;
- Naive Bayes + TF-IDF: 66,58%.

Por outro lado, na Regressão Logística ocorreu o comportamento contrário:

- Regressão Logística + CountVectorizer: 67,72%;
- Regressão Logística + TF-IDF: 73,19%.

Assim, para este corpus, o TF-IDF foi mais eficiente quando combinado com a Regressão Logística, enquanto o CountVectorizer apresentou melhor resultado quando utilizado com Naive Bayes.

Isso demonstra a importância de testar diferentes combinações entre representação dos dados e algoritmo de classificação, pois o desempenho não depende apenas do modelo utilizado.

---

## 13. Comparação Geral dos Modelos

Considerando a acurácia, a ordem de desempenho dos experimentos foi:

1. Regressão Logística + TF-IDF — 73,19%;
2. Naive Bayes + CountVectorizer — 72,40%;
3. Regressão Logística + CountVectorizer — 67,72%;
4. Naive Bayes + TF-IDF — 66,58%.

Portanto, o modelo que apresentou o melhor desempenho geral foi a Regressão Logística utilizando TF-IDF.

A escolha do modelo final, entretanto, não deve considerar somente a acurácia. Como o problema envolve duas classes e existe desbalanceamento, também é necessário observar precision, recall e F1-score.

Nesse sentido, o modelo vencedor apresentou F1-score de 0,57 para a classe 1, mostrando que ainda existe espaço para melhorias na identificação dessa classe.

---

## 14. Matrizes de Confusão

As matrizes de confusão foram utilizadas para analisar o desempenho dos quatro experimentos, permitindo identificar a quantidade de classificações corretas e incorretas para cada classe. Nas matrizes apresentadas, as linhas representam as classes verdadeiras e as colunas representam as classes previstas pelos modelos.

Os valores da diagonal principal correspondem às classificações corretas, enquanto os valores fora da diagonal representam os erros cometidos pelos classificadores.

### 14.1 Naive Bayes com Bag-of-Words (CountVectorizer)

A matriz de confusão apresentou os seguintes resultados:

| Classe verdadeira | Prevista 0 | Prevista 1 |
|-------------------|------------|------------|
| Classe 0          | 587        | 189        |
| Classe 1          | 124        | 234        |

O modelo classificou corretamente 587 exemplos da classe 0 e 234 exemplos da classe 1. Por outro lado, 189 exemplos da classe 0 foram classificados incorretamente como classe 1, enquanto 124 exemplos da classe 1 foram classificados como classe 0.

Esses resultados demonstram que o modelo apresentou bom desempenho na identificação da classe 0, mas também conseguiu reconhecer uma parcela relevante dos exemplos da classe 1. A quantidade de falsos negativos da classe 1 foi de 124, indicando que alguns textos pertencentes a essa classe não foram identificados corretamente.

## 14.2 Naive Bayes com TF-IDF

A matriz de confusão do Naive Bayes utilizando TF-IDF apresentou os seguintes valores:

| <b>Classe verdadeira</b> | <b>Prevista 0</b> | <b>Prevista 1</b> |
|--------------------------|-------------------|-------------------|
| Classe 0                 | 494               | 282               |
| Classe 1                 | 97                | 261               |

Nesse experimento, o modelo classificou corretamente 494 exemplos da classe 0 e 261 exemplos da classe 1. Foram registrados 282 falsos positivos e 97 falsos negativos.

Em comparação com o Naive Bayes utilizando CountVectorizer, o uso de TF-IDF aumentou a quantidade de exemplos da classe 1 identificados corretamente, de 234 para 261, e reduziu os falsos negativos de 124 para 97.

Entretanto, o número de falsos positivos aumentou de 189 para 282. Isso indica que o modelo passou a classificar mais exemplos da classe 0 incorretamente como pertencentes à classe 1. Portanto, embora tenha apresentado maior capacidade de identificar a classe minoritária, essa melhoria ocorreu acompanhada de um aumento nos erros de classificação da classe majoritária.

## 14.3 Regressão Logística com Bag-of-Words (CountVectorizer)

A matriz de confusão da Regressão Logística com CountVectorizer apresentou os seguintes resultados:

| <b>Classe verdadeira</b> | <b>Prevista 0</b> | <b>Prevista 1</b> |
|--------------------------|-------------------|-------------------|
| Classe 0                 | 540               | 236               |
| Classe 1                 | 130               | 228               |

O modelo classificou corretamente 540 exemplos da classe 0 e 228 exemplos da classe 1. Além disso, foram observados 236 falsos positivos e 130 falsos negativos.

Os resultados mostram que a Regressão Logística com CountVectorizer conseguiu identificar corretamente uma quantidade considerável de exemplos das duas classes.

Entretanto, os 130 falsos negativos indicam que ainda houve dificuldade para reconhecer parte dos textos pertencentes à classe 1.

Quando comparado ao Naive Bayes com CountVectorizer, esse modelo apresentou menos verdadeiros negativos e menos verdadeiros positivos, demonstrando desempenho inferior nessa combinação de representação e algoritmo.

## 14.4 Regressão Logística com TF-IDF

A matriz de confusão da Regressão Logística com TF-IDF apresentou os seguintes valores:

| <b>Classe verdadeira</b> | <b>Prevista 0</b> | <b>Prevista 1</b> |
|--------------------------|-------------------|-------------------|
| Classe 0                 | 629               | 147               |
| Classe 1                 | 157               | 201               |

Nesse experimento, o modelo classificou corretamente 629 exemplos da classe 0 e 201 exemplos da classe 1. Foram registrados 147 falsos positivos e 157 falsos negativos.

O principal destaque foi a quantidade de exemplos da classe 0 identificados corretamente, que foi a maior entre os quatro experimentos. Além disso, esse modelo apresentou o menor número de falsos positivos, com 147 ocorrências.

Entretanto, foram observados 157 falsos negativos, indicando que uma parcela dos textos pertencentes à classe 1 foi classificada incorretamente como classe 0. Dessa forma, apesar de apresentar a maior acurácia geral nos resultados do experimento, o modelo ainda apresenta limitações na identificação da classe minoritária.

## 14.5 Comparação das Matrizes de Confusão

A comparação das quatro matrizes permite observar diferenças importantes entre os modelos.

O Naive Bayes com TF-IDF apresentou o maior número de verdadeiros positivos da classe 1, com 261 exemplos corretamente identificados, além do menor número de falsos negativos, com 97 ocorrências. Isso demonstra maior capacidade de recuperar os exemplos da classe 1.

Por outro lado, a Regressão Logística com TF-IDF apresentou o maior número de classificações corretas da classe 0, com 629 exemplos, e o menor número de falsos positivos, com 147 ocorrências.

O Naive Bayes com CountVectorizer apresentou 587 classificações corretas da classe 0 e 234 da classe 1, enquanto a Regressão Logística com CountVectorizer apresentou 540 e 228 classificações corretas, respectivamente.

Portanto, a análise das matrizes evidencia que o melhor modelo depende também do objetivo da classificação. Quando se busca identificar a maior quantidade possível de exemplos da classe 1, o Naive Bayes com TF-IDF apresentou o resultado mais favorável. Entretanto, quando se considera a acurácia geral e o reconhecimento da classe 0, a Regressão Logística com TF-IDF apresentou o melhor desempenho geral nos resultados consolidados do experimento.

Esses resultados reforçam a importância de analisar conjuntamente as matrizes de confusão e as métricas de avaliação, evitando escolher um modelo apenas com base na acurácia.

## 15. Discussão Crítica

Os resultados mostram que a combinação entre algoritmo e representação das características possui influência significativa no desempenho da classificação.

A Regressão Logística com TF-IDF obteve o melhor resultado geral, alcançando 73,19% de acurácia. Esse comportamento pode estar relacionado à capacidade do TF-IDF de atribuir maior peso a termos mais discriminativos e à capacidade da Regressão Logística de trabalhar eficientemente com representações esparsas.

O Naive Bayes com CountVectorizer também apresentou desempenho competitivo, alcançando 72,40% de acurácia. Esse resultado confirma que métodos mais simples podem apresentar resultados satisfatórios em tarefas de classificação textual.

Entretanto, os resultados também evidenciam uma dificuldade na classificação da classe 1. Em todos os experimentos, os valores de precision e F1-score dessa classe ficaram abaixo dos obtidos para a classe 0.

Isso pode estar relacionado tanto ao desbalanceamento original dos dados quanto à própria dificuldade de diferenciar linguisticamente determinadas mensagens. Mesmo com a utilização do SMOTE, o balanceamento durante o treinamento não garante que as duas classes sejam igualmente fáceis de distinguir.

---

## 16. Limitações do Projeto

Uma das principais limitações encontradas foi o desbalanceamento original entre as classes. A classe 0 possui mais que o dobro da quantidade de exemplos da classe 1.

Outra limitação está relacionada ao tamanho do corpus. Embora 5.670 textos sejam suficientes para realizar experimentos de classificação, problemas de linguagem natural

podem apresentar grande diversidade linguística, e um corpus maior e mais diversificado poderia contribuir para melhorar a generalização dos modelos.

Também existe a possibilidade de que determinadas expressões apresentem diferentes significados dependendo do contexto. Modelos baseados apenas na frequência ou peso das palavras podem apresentar dificuldades para compreender ironia, sarcasmo, ambiguidades e relações semânticas mais complexas.

Além disso, os modelos utilizados são métodos tradicionais de aprendizado de máquina e não exploram diretamente representações semânticas profundas da linguagem.

---

## 17. Melhorias Possíveis

Diversas melhorias poderiam ser implementadas em uma versão futura do projeto.

No dataset, seria interessante aumentar a quantidade de exemplos, principalmente da classe minoritária, além de revisar possíveis duplicidades, inconsistências e erros de anotação.

No pré-processamento, poderiam ser testadas diferentes estratégias de remoção de stopwords, stemming e lematização, verificando experimentalmente qual combinação apresenta melhor desempenho.

Também poderiam ser testadas diferentes configurações do TF-IDF, incluindo unigramas e bigramas, além de ajustes nos parâmetros dos classificadores.

Outra melhoria seria realizar validação cruzada, permitindo avaliar a estabilidade dos resultados em diferentes divisões dos dados.

Também seria possível utilizar técnicas específicas para lidar com o desbalanceamento, comparando o SMOTE com outras estratégias, como pesos de classe e métodos de amostragem.

---

## 18. Extensão para Aplicações Reais

Em uma aplicação real, o sistema poderia ser utilizado como uma ferramenta de apoio à identificação automática de possíveis discursos de ódio em plataformas digitais.

Um sistema desse tipo poderia analisar novas mensagens em tempo real e atribuir uma probabilidade de pertencimento às classes. As mensagens classificadas como potencialmente problemáticas poderiam ser encaminhadas para uma segunda etapa de análise humana.

Entretanto, a utilização em um cenário real exigiria maior preocupação com falsos positivos e falsos negativos. Uma classificação incorreta poderia resultar na remoção indevida de um conteúdo ou, no sentido contrário, permitir que um conteúdo ofensivo não fosse identificado.

Por isso, o modelo não deveria ser utilizado isoladamente para decisões críticas. O ideal seria utilizá-lo como uma ferramenta de apoio à moderação humana.

---

## 19. Técnicas Avançadas para uma Segunda Versão

Em uma segunda versão do projeto poderiam ser utilizadas técnicas mais avançadas de Processamento de Linguagem Natural.

Uma possibilidade seria utilizar redes neurais recorrentes, como LSTM, capazes de considerar a sequência das palavras nos textos.

Outra alternativa seria utilizar arquiteturas baseadas em Transformers, como BERT e modelos pré-treinados para língua portuguesa. Essas abordagens conseguem trabalhar com representações contextuais das palavras e podem apresentar vantagens em tarefas em que o significado depende do contexto.

Também poderiam ser testadas técnicas de embeddings de palavras, como Word2Vec ou FastText, para representar relações semânticas entre os termos.

Além disso, seria possível realizar ajuste de hiperparâmetros, validação cruzada e comparação entre diferentes arquiteturas para identificar uma configuração mais adequada ao problema.

---

## 20. Conclusão

O projeto permitiu desenvolver um pipeline completo de Processamento de Linguagem Natural aplicado à classificação de textos em língua portuguesa.

Inicialmente, foi realizada a preparação dos dados e a análise exploratória do corpus. O conjunto utilizado possui 5.670 textos distribuídos em duas classes, sendo 3.882 exemplos da classe 0 e 1.788 da classe 1. A diferença entre as classes demonstrou a existência de desbalanceamento, sendo utilizado o SMOTE durante o treinamento para reduzir esse problema.

Na etapa de classificação foram comparados Naive Bayes e Regressão Logística utilizando duas representações textuais: CountVectorizer e TF-IDF. Os resultados demonstraram que

a melhor combinação foi a Regressão Logística com TF-IDF, que apresentou acurácia de aproximadamente 73,19%.

O segundo melhor resultado foi obtido pelo Naive Bayes com CountVectorizer, com aproximadamente 72,40% de acurácia. Já a Regressão Logística com CountVectorizer apresentou 67,72%, enquanto o Naive Bayes com TF-IDF alcançou 66,58%.

A comparação também mostrou que o desempenho depende da combinação entre representação e algoritmo. O TF-IDF apresentou melhor resultado com a Regressão Logística, enquanto o CountVectorizer foi mais eficiente com o Naive Bayes.

Apesar dos resultados, ainda existem limitações importantes, principalmente relacionadas à dificuldade de classificação da classe 1. Os valores de F1-score demonstram que essa classe continua sendo mais difícil de identificar corretamente.

## **Respostas às questões finais**

### **1. O que mais dificultou o processo?**

A principal dificuldade foi trabalhar com textos em linguagem natural, pois as mensagens apresentam diferentes formas de escrita, ruídos e possíveis ambiguidades. Além disso, o desbalanceamento entre as classes dificultou a identificação da classe minoritária.

### **2. Quais melhorias poderiam ser implementadas no dataset, no pré-processamento ou nos modelos?**

Seria possível aumentar o dataset, melhorar a qualidade das anotações, testar diferentes técnicas de pré-processamento, ajustar os parâmetros dos modelos e utilizar métodos alternativos de balanceamento. Também seria interessante utilizar validação cruzada e testar diferentes configurações do TF-IDF.

### **3. Como você estenderia este projeto para aplicações reais?**

O projeto poderia ser transformado em um sistema de apoio à moderação de conteúdos, capaz de analisar novas mensagens e identificar automaticamente aquelas com maior probabilidade de pertencer à classe de discurso de ódio. As classificações poderiam posteriormente ser avaliadas por moderadores humanos.

### **4. Quais técnicas avançadas poderiam ser aplicadas em uma segunda versão?**

Poderiam ser utilizadas técnicas de embeddings, redes neurais LSTM, modelos Transformers e modelos pré-treinados para língua portuguesa, como BERT. Também poderiam ser realizados ajustes de hiperparâmetros e validação cruzada para melhorar a capacidade de generalização.

Conclui-se, portanto, que o experimento demonstrou a viabilidade da utilização de técnicas tradicionais de aprendizado de máquina para classificação de textos e mostrou que a escolha da representação das características possui papel fundamental no desempenho final do modelo.